Høgskulen på Vestlandet

DAT107 - Databaser

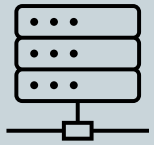
Video 16: MongoDB Query API

Introduksjon til NoSQL

Dag Larsen
Oppdatert av Lasse Jenssen

Læringsmål

Vi ser litt nærmere på ulike klienter



MongoDB Query API

CRUD operasjoner

Primær Nøkkel (_id) & datatypen **ObjectId()**

Litteratur

NoSQL

Læreboka:

- ✓ Kapittel 15.4 - MongoDB
- ✓ Foilene (pdf) benyttet i denne videoen

Høgskulen på Vestlandet

Primær Nøkkel
&
ObjectId

MongoDB

Nøkkel "_id"

Primærnøkkel i MongoDB

- ✓ **Obligatorisk: En unik "_id" nøkkel** som fungerer som en **primærnøkkel** for aktuell "collection".
- ✓ **Alltid første felt i et document**.
- ✓ **Hvis mangler** => MongoDB vil automatisk generere en ObjectId og legge til i et _id felt.
- ✓ MongoDB vil alltid opprette **en unik indeks** på "_id" feltet under opprettelsen av en "collection".
- ✓ Kan inneholde verdier av hvilken som helst BSON data type, med unntak av en "array", "regex" eller "undefined".

```
> db.eiere.insertOne( { navn: "Kari", by: "Tromsø", alder: 25 } );
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6772ddd31811d14cb7d4b0c6') }
}
```

Høgskulen på Vestlandet

Primærnøkkel : "_id" & ObjectId

CRUD Operasjoner

```
> var myId = ObjectId();
> print("ID: " + myId);
ID: 677291161811d14cb7d4b0c3

> db.eiere.insertOne({navn: "Ole"});
{
  acknowledged: true,
  insertedId: ObjectId('6772914b1811d14cb7d4b0c4')
}

> db.eiere.insertOne({ _id: 1, navn: "Ole"});
{ acknowledged: true, insertedId: 1 }

> db.eiere.insertOne({ _id: 1, navn: "Per"});
MongoServerError: E11000 duplicate key error collection: lager.eiere index:
_id_ dup key: { _id: 1 }
```

```
> db.eiere.find();
[{ _id: ObjectId('...d4b0c4'),
          navn: 'Ole' },
 { _id: 1, navn: 'Ole' }]
```

Ikke anbefalt å blande datatyper

Høgskulen på Vestlandet

Nøkkel "_id"

Alternativer for _id nøkkelen

- ✓ MongoDB bruker default en ObjectId
- ✓ Hvis tilgjengelig kan man bruke en natural unique identifier
  - Sparer plass, og man unngår en ekstra indeks
  - Man kan bruke auto-incrementing tall.
  - Man kan lage en UUID i applikasjons koden.
    (og eventuelt lagre UUID som en verdi med BSON BinData type)

Høgskulen på Vestlandet

ObjectId() vs ObjectId

Metode versus Objekt

Metode: ObjectId()

- ✓ Returnerer en ny 12-byte ObjectId

Object: ObjectID

- ✓ Består av:
  - **Tidsstempel**: De første 4 byte representerer tidsstempelet for når ObjectId ble opprettet, i sekunder siden Unix epoch.
  - **Maskin-ID:** De neste 3 byte er en unik identifikator for maskinen som genererte ObjectId.
  - **Proses-ID**: De påfølgende 2 byte representerer prosess-ID-en til prosessen som genererte ObjectId.
  - **Ziffer**: De siste 3 byte er et tilfeldig generert nummer, som sikrer at ObjectId-er generert på samme maskin og i samme prosess er unike.

Primary Key & ObjectId

Et eksempel

```
> db.eiere.insertMany(
[
  { navn: "Kari", by: "Tromsø", alder: 25 },
  { navn: "Per", by: "Oslo", alder: 70 },
  { navn: "Lise", by: "Bergen", alder: 55 },
  { navn: "Mona", by: "Trondheim" },
  { navn: "Ole", by: "Oslo", alder: 77 }
]);
```

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6772ddd31811d14cb7d4b0c6'),
    '1': ObjectId('6772ddd31811d14cb7d4b0c7'),
    '2': ObjectId('6772ddd31811d14cb7d4b0c8'),
    '3': ObjectId('6772ddd31811d14cb7d4b0c9'),
    '4': ObjectId('6772ddd31811d14cb7d4b0ca')
  }
}
```

Date Maskin Ziffer

Prosess

Høgskulen på Vestlandet

ObjectId: First 4 byte

Unix Epoch Time

Tidsstempel:

- ✓ De første 4 byte representerer tidsstempelet for når ObjectId ble opprettet, i sekunder, siden Unix epoch.

```
> var myId = ObjectId();
> print("ID: " + myId);
ID: 6776d6ad10d7a0c858d4b0c3

> print("Time: " + myId.getTimestamp());
Time: Thu Jan 02 2025 18:10:53 GMT+0000

4 byte:   01100111 01110110 11010110 10101101
Hex:      67       76.      d6       ad

Decimal:  1735841453 sekunder + 1.januar 1970
                     = 02.01.2025 18:10:53 (GMT)
```

Høgskulen på Vestlandet

Posisjon (fra høyre)	Potens	Verdi
0	2^0	1
2	2^2	4
3	2^3	8
5	2^5	32
7	2^7	128
9	2^9	512
10	2^{10}	1 024
12	2^{12}	4 096
14	2^{14}	16 384
15	2^{15}	32 768
17	2^{17}	131 072
18	2^{18}	262 144
20	2^{20}	1 048 576
21	2^{21}	2 097 152
22	2^{22}	4 194 304
24	[illegible]	[illegible]
25		
26		
29		
30		
Total sum		[illegible]

Create
Read
Update
Delete

Mongo Commands

CRUD Operasjoner

MongoDB Query API

Operasjon	Beskrivelse	MongoDB - kommando
Create	Legge til nytt dokument (et eller flere)	db.collection.**insertOne**({ ... }) db.collection.**insertMany**([{ ... }, { ... }])
Read	Hente dokumenter	db.collection.**find**({ kriterier }) db.collection.**findOne**({ kriterier })
Update	Oppdatere eksisterende dokument(er)	db.collection.**updateOne**({ filter }, { $set: { felt: verdi } }) db.collection.**updateMany**({ filter }, { $set: { felt: verdi } })
Delete	Slette dokument(er)	db.collection.**deleteOne**({ filter }) db.collection.**deleteMany**({ filter })

Høgskulen på Vestlandet

Create

Mongo Commands

db.<collection>.insertOne

CRUD Operasjoner: Create

Collection må ikke eksistere. Opprettes.

```
> db.eiere.insertOne(
  { navn: "Kari", by: "Tromsø", alder: 25 }
);
```

Betyr at MongoDB bekrefter at data er blitt lagt til.

```
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6772ddd31811d14cb7d4b0c6') }
}
```

Høgskulen på Vestlandet

db.<collection>.insertOne – WriteConcern: «w:0»

CRUD Operasjoner: Create

- ✓ **Ikke lov i transaksjoner**: Unacknowledged writes kan ikke brukes i transaksjoner. Default: { w:1 }
- ✓ **w:0 kan gi lavere latency.**

```
> db.eiere.insertOne(
  { navn: "Kari", by: "Tromsø", alder: 25 },
  { writeConcern: { w: 0 } }
);
```

Risikerer at INSERT ikke blir gjennomført.

```
{
  acknowledged: false,
}
```

Sendt. Men ikke bekreftet.

Konsekvens: ObjectId kan ikke returneres.
(Eventuelt: Generer på klientsiden, og sett inn)

Høgskulen på Vestlandet

db.<collection>.insertMany

CRUD Operasjoner: Create

```
> db.eiere.insertMany(
    [
      { navn: "Kari", by: "Tromsø", alder: 25 },
      { navn: "Per",  by: "Oslo",   alder: 70 },
      { navn: "Lise", by: "Bergen", alder: 55 }
    ]
);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6772ddd31811d14cb7d4b0c6’),
    '1': ObjectId('6772ddd31811d14cb7d4b0c7’),
    …
  }
}
```

Høgskulen på Vestlandet

Read

Mongo Commands

db.<collection>.find

CRUD Operasjoner: Read

```
hvl-db> db.eiere.find({ navn: "Kari" })

[ { _id: ObjectId('696248f52a95fa23a71c5d02'), navn: 'Kari' } ]

hvl-dba> db.eiere.find({ navn: "kari" })

[]

hvl-dba> db.eiere.find({ navn: /karl/i })

[ { _id: ObjectId('696248f52a95fa23a71c5d02'), navn: 'Kari' } ]

hvl-dba> db.eiere.find({ navn: /^k/i })

hvl-dba> db.eiere.find({ navn: /k$/i })
```

Case sensitive

Case insensitive

Starts with «k»

Ends with «k»

Høgskulen på Vestlandet

db.<collection>.find & $regex

CRUD Operasjoner: Read

```
> db.eiere.find({ navn: { $regex: /pattern/, $options: '<options>' }});
> db.eiere.find({ navn: { $regex: /kari/, $options: 'i' }})
[ { _id: ObjectId('696248f52a95fa23a71c5d02'), navn: 'Kari' } ]

> db.eiere.find({ navn: { $regex: /pattern/<options> }});
> db.eiere.find({ navn: { $regex: /kari/i }})
[ { _id: ObjectId('696248f52a95fa23a71c5d02'), navn: 'Kari' } ]

> db.eiere.find({ navn: /pattern/<options> });
> db.eiere.find({ navn: /kari/i })
[ { _id: ObjectId('696248f52a95fa23a71c5d02'), navn: 'Kari' } ]
```

Høgskulen på Vestlandet

db.<collection>.find & $regex

CRUD Operasjoner: Read

- ✓ **Oppgave:** Lag en spørring som finner alle som har navn som starter på «k», og slutter på «i» (case insensitivt)

```
> db.eiere.find({ navn: /^k.*i$/i })
```

```
[ { _id: ObjectId('696248f52a95fa23a71c5d02'), navn: 'Kari' } ]
```

Høgskulen på Vestlandet

db.<collection>.find & $regex

CRUD Operasjoner: Read

- ✓ **Oppgave:** Lag en spørring som finner alle som har navn med «a» i andre bokstav (case insensitivt).

```
> db.eiere.find({ navn: /^[a-å]a/i })
```

```
[
  {
    _id: ObjectId('6962a7aa31340bee453f118e'),
    navn: 'Kari',
    by: 'Oslo'
  },
  {
    _id: ObjectId('6962a7aa31340bee453f1191'),
    navn: 'Tanja',
    by: 'Oslo'
  }
]
```

Operatoren $not

CRUD Operasjoner: Read

✓ **Oppgave:** Lag en spørring som finner alle som IKKE har navn med «a» i andre bokstav (case insensitivt).

```
> db.eiere.find({ navn: { $not: /^[a-å]a/i }})
```

```
[
  {
    _id: ObjectId('6962a7aa31340bee453f118f'),
    navn: 'Nils',
    by: 'Trondheim'
  },
  {
    _id: ObjectId('6962a7aa31340bee453f1190'),
    navn: 'Trond',
    by: 'Tromsø'
  }
]
```

Flere filtre og projeksjon

CRUD Operasjoner: Read

```
> db.eiere.find({ navn: /^Trond/, by: "Tromsø" });
[{
   _id: ObjectId('6962a7aa31340bee453f118f'),
   navn: 'Trond',
   by: 'Tromsø'
}]

> db.eiere.find({ by: "Tromsø" }, { navn:1 });
[{ _id: ObjectId('6962a7aa31340bee453f1190'), navn: 'Trond' }]

> db.eiere.find({ by: "Tromsø" }, { _id: 0, navn:1 });
[{ navn: 'Trond' }]
```

Høgskulen på Vestlandet

Update

Mongo Commands

db.<collection>.updateOne

CRUD Operasjoner: Update

```
> db.eiere.updateOne(
   { _id: ObjectId('676d5f590692e05cb3d1c46e')},
   { $set:  { by: "Oslo" } }
  );
```

Filter

Update

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Høgskulen på Vestlandet

db.<collection>.replaceOne

CRUD Operasjoner: Update

```
> db.eiere.find({ _id: ObjectId('6962a7aa31340bee453f1191')})
[ { _id: ObjectId('696248f5..'), navn: 'Kari', by: 'Oslo' } ]

> db.eiere.replaceOne(
    { _id: ObjectId('6962a7aa31340bee453f1191')},
    { navn: "Lise" }
  )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

> db.eiere.find({ _id: ObjectId('6962a7aa31340bee453f1191')})
[ { _id: ObjectId('696248f5..'), navn: 'Lise' } ]
```

Filter

Replaced with

updateOne & Upsert

CRUD Operasjoner: Update

```
> db.eiere.updateOne(
    { navn: "Karl" },
    { $setOnInsert: { by: "Oslo" } },
    { upsert: true }
  )
{
  acknowledged: true,
  insertedId: ObjectId('676d6706f095956e7f564b57'),
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
}
```

Eksisterer ikke

Filter

Update

Options

Gir kun mening hvis upsert: true

```
> db.eiere.find({ navn: "Karl" })
[{ _id: ObjectId('676d670..'), navn: 'Karl', by: 'Oslo' }]
```

Høgskulen på Vestlandet

updateOne & Upsert

CRUD Operasjoner: Update

```
> db.eiere.updateOne(
    { navn: "Karl" },
    { $setOnInsert: { alder: 50 } },
    { upsert: true }
  )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
```

Eksisterer nå

$setOnInsert

✓ **Ingen endring. Hvorfor?**

```
> db.eiere.find({ navn: "Karl" })
[{ _id: ObjectId('676d670..'), navn: 'Karl', by: 'Oslo' }]
```

Høgskulen på Vestlandet

updateOne & Upsert

CRUD Operasjoner: Update

```
> db.eiere.updateOne(
    { navn: "Karl" },
    { $set: { alder: 51 } },
    { upsert: true }
  )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Eksisterer fremdeles

$set

- ✓ Blir brukt (set) ved både
  - Update
  - Insert

- ✓ Nå ble det endring. Hvorfor?

```
> db.eiere.find({ navn: "Karl" })
[ { _id: ObjectId('696248f5..'), navn: 'Karl', alder: 51 } ]
```

Høgskulen på Vestlandet

updateOne & Upsert

CRUD Operasjoner: Update

```
> db.eiere.updateOne(
    { navn: "Carl" },
    { $set: { alder: 26 } },
    { upsert: false }
  )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
> db.eiere.find({ navn: "Carl" })
[]
```

Eksisterer **ikke**

Default

✓ **Nå ble IKKE det endring. Hvorfor?**

Høgskulen på Vestlandet

updateOne & Unset

CRUD Operasjoner: Update

```
> db.eiere.updateOne(
    { navn: "Karl" },
    { $unset: { alder: 1 } }
    { upsert: false }
  )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Eksisterer fremdeles

✓ Fjerner et nøkkel-verdi par

$unset

✓ Nå ble det endring. Hvorfor?

```
> db.eiere.find({ navn: "Karl" })
[ { _id: ObjectId('696248f5..'), navn: 'Karl' } ]
```

Høgskulen på Vestlandet

Delete

Mongo Commands

db.<collection>.deleteOne

CRUD Operasjoner: Delete

```
> db.eiere.deleteOne(
    { _id: ObjectId('676d5f590692e05cb3d1c46e')}
);
```

Filter

```
{ acknowledged: true, deletedCount: 1 }
```

Høgskulen på Vestlandet

db.<collection>.deleteMany

CRUD Operasjoner: Delete

✓ **Oppgave:** Lag en spørring som sletter alle eiere i «Oslo».

```
> db.eiere.deleteMany({ by: "Oslo" } );
```

Filter

```
{ acknowledged: true, deletedCount: 2 }
```

Høgskulen på Vestlandet

Read:
Flere filtre

Mongo Commands

CRUD – operssjoner fortsetter

Eksempel data

```
db.eiere.insertMany(
[
  { navn: 'Kari', by: 'Oslo' },
  { navn: 'Tanja', by: 'Oslo' },
  { navn: 'Nils', by: 'Trondheim' },
  { navn: 'Trond', by: 'Tromsø' },
  { navn: 'Eva', by: 'Oslo'}
]
);
```

Høgskulen på Vestlandet

db.<collection>.find()

CRUD: Projeksjon uten filter

```
> db.eiere.find({}, { navn:1 })
[
  { _id: ObjectId('6962a7aa31340bee453f1190'), navn: 'Trond' },
  { _id: ObjectId('6962cf2031340bee453f1192'), navn: 'Kari' },
  { _id: ObjectId('6962cf2031340bee453f1193'), navn: 'Tanja' },
  { _id: ObjectId('6962cf2031340bee453f1194'), navn: 'Nils' }
]
```

```
> db.eiere.findOne({}, { navn:1 })
{ _id: ObjectId('6962a7aa31340bee453f1190'), navn: 'Trond' }
```

Høgskulen på Vestlandet

Metoder: size(), count() & limit()

CRUD Operasjoner: Read Aggregates & Limit

```
> db.eiere.find({ by: "Oslo" }).size()
3

> db.eiere.find({ by: "Oslo" }).count()
3
```

cursor.size() gir deg antall dokumenter som er tilgjengelige for iterasjon i cursoren.

cursor.count() gir deg det totale antallet dokumenter som matcher forespørselen.

```
> db.eiere.find({ by: "Oslo" },{navn:1}).limit(2)
[
  { _id: ObjectId('6962d5ce31340bee453f1196'), navn: 'Kari' },
  { _id: ObjectId('6962d5ce31340bee453f1197'), navn: 'Tanja' }
]
```

Metode: sort()

CRUD Operasjoner: Sorter stigende

```
> db.eiere.find({ by: "Oslo" },{navn:1})

[
  { _id: ObjectId('6962d5ce31340bee453f1196'), navn: 'Kari' },
  { _id: ObjectId('6962d5ce31340bee453f1197'), navn: 'Tanja' },
  { _id: ObjectId('6962d5ce31340bee453f119a'), navn: 'Eva' }
]
```

```
> db.eiere.find({ by: "Oslo" },{navn:1}).sort({navn: 1})

[
  { _id: ObjectId('6962d5ce31340bee453f119a'), navn: 'Eva' },
  { _id: ObjectId('6962d5ce31340bee453f1196'), navn: 'Kari' },
  { _id: ObjectId('6962d5ce31340bee453f1197'), navn: 'Tanja' }
]
```

Metode: sort()

CRUD Operasjoner: Sorter synkende

```
> db.eiere.find({ by: "Oslo" },{navn:1}).sort({navn: -1})

[
  { _id: ObjectId('6962d5ce31340bee453f1197'), navn: 'Tanja' },
  { _id: ObjectId('6962d5ce31340bee453f1196'), navn: 'Kari' },
  { _id: ObjectId('6962d5ce31340bee453f119a'), navn: 'Eva' }
]
```

Høgskulen på Vestlandet

Hva forteller dette oss?

CRUD Operasjoner: Sort & Limit

```
> db.eiere.find({ by: "Oslo" },{navn:1}).limit(2)
[
  { _id: ObjectId('6962d5ce31340bee453f1196'), navn: 'Kari' },
  { _id: ObjectId('6962d5ce31340bee453f1197'), navn: 'Tanja' }
]

> db.eiere.find({ by: "Oslo" },{navn:1}).limit(2).sort({navn: 1})
[
  { _id: ObjectId('6962d5ce31340bee453f119a'), navn: 'Eva' },
  { _id: ObjectId('6962d5ce31340bee453f1196'), navn: 'Kari' }
]
```

Høgskulen på Vestlandet

Hva forteller dette oss – del2?

CRUD Operasjoner: Sort

```
> db.eiere.find({ by: "Oslo" },{navn:1}).sort({navn: -1}).limit(2)
[
  { _id: ObjectId('6962d5ce31340bee453f1197'), navn: 'Tanja' },
  { _id: ObjectId('6962d5ce31340bee453f1196'), navn: 'Kari' }
]

> db.eiere.find({ by: "Oslo" },{navn:1}).limit(2).sort({navn: -1})
[
  { _id: ObjectId('6962d5ce31340bee453f1197'), navn: 'Tanja' },
  { _id: ObjectId('6962d5ce31340bee453f1196'), navn: 'Kari' }
]
```

Årsak: sort() utføres før limit()

Høgskulen på Vestlandet

Sammenlignings operatorer

CRUD Operasjoner

Name	Description
`$eq`	Matches values that are equal to a specified value.
`$gt`	Matches values that are greater than a specified value.
`$gte`	Matches values that are greater than or equal to a specified value.
`$in`	Matches any of the values specified in an array.
`$lt`	Matches values that are less than a specified value.
`$lte`	Matches values that are less than or equal to a specified value.
`$ne`	Matches all values that are not equal to a specified value.
`$nin`	Matches none of the values specified in an array.

Sammenlignings operatorer

CRUD Operasjoner: : $eq, $ne, $in, $nin, $gt, $gte, $lt & $lte

```
> db.eiere.find({by:{$eq:"Oslo"}})
> db.eiere.find({by:"Oslo"})
```

Equal

```
> db.eiere.find({by:{$ne:"Oslo"}})
```

Not Equal

```
> db.eiere.find({by:{$in:["Oslo", "Bergen"]}})
> db.eiere.find({by:{$nin:["Oslo", "Bergen"]}})
```

IN & NOT IN

```
> db.eiere.find({alder: {$gt:18})
> db.eiere.find({alder: {$gte:18})
```

Greater than & greater than or equal

```
> db.eiere.find({alder: {$lt:18})
> db.eiere.find({alder: {$lte:18})
```

Less than & less than or equal

Logiske operatorer

CRUD Operasjoner: $and, $or, $not, $ne & $nor

```
> db.eiere.find({$and:[{by:{$eq:"Oslo"}},{alder: {$lt:20}}]})
```

AND

```
> db.eiere.find({$or:[{by:{$eq:"Oslo"}},{alder: {$lt:20}}]})
```

OR

```
> db.eiere.find({by:{$not:{$eq:"Oslo"}}})
> db.eiere.find({by:{$ne:"Oslo"}})
```

NOT EQUAL TO

```
> db. eiere.find({ $nor:[ {by:"Oslo"},{alder: {$gte: 10}} ] })
```

NOR

Oppsummering

MongoDB Query API

Høgskulen på Vestlandet

- ✓ MongoDB Query API
- ✓ Primærnøkkel (_id) & ObjectId datatypen
- ✓ CRUD-operasjoner
  - db.<samling>.insert<One|Many>
  - db.<samling>.find()
  - db.<samling>.findOne()
  - db.<samling>.update<One|Many>
  - db.<samling>.delete<One|Many>

- ✓ updateOne(
  { <**filter**> },
  { <**update**> },
  { <**options**>}
)
- ✓ $set vs. $setOnInsert
- ✓ **upsert**: true | false

Oppsummering

Vi har sett på ...

Høgskulen på Vestlandet

- ✓ Metoder:
  - size() - antall tilgjengelig i cursor
  - count() - totalt antall
  - limit() - begrense antall i retur
  - sort() - sortere dokumenter
- ✓ Logiske operatorer
  - $and - begge uttrykk på være sanne
  - $or - et av uttrykkene må være sant
  - $not - ikke
  - $ne - ikke lik
  - $nor - hverken
- ✓ Sammenligning operatorer
  - $eq - lik
  - $ne - ikke lik
  - $in - i (mengden)
  - $nin - ikke i (mengden)
  - $gt - større enn
  - $gte - større enn eller lik
  - $lt - mindre ennd
  - $lte - mindre enn eller li

DEMO